Domain-Adversarial Deep Learning for Cross-Dataset Cardiac Arrhythmia Detection

Aaron Tsui[1,†]

[1] Northwestern University School of Professional Studies

Master of Science in Data Science Program

633 Clark St, Evanston, IL 60208

† Address to which correspondence should be addressed:

aaron.tsui.careers@gmail.com

www.linkedin.com/in/aaron-tsui/

Abstract

Accurate and generalizable arrhythmia detection from electrocardiogram (ECG) signals remains a core challenge in clinical cardiology, particularly when models are deployed across diverse healthcare institutions with heterogeneous data sources. Accurate detection is also essential for early intervention and reduction of adverse cardiac events. This study develops and evaluates two convolutional neural network (CNN) architectures for arrhythmia classification using two primary datasets: PTB-XL, a large 12-lead clinical ECG corpus, and the Lobachevsky University Electrocardiography Database, which provides additional signal diversity across patient populations and acquisition conditions. The first CNN model was adapted from a prior study, while the second CNN was designed to yield improved discrimination of arrhythmia classes through deeper convolutional blocks and enhanced regularization. Model performance was assessed using confusion matrices, receiver operating characteristic (ROC) curves, area-under-the-curve (AUC) scores, and precision-recall curves. Results demonstrate that the second CNN achieves superior class separation and robustness across datasets, achieving a test accuracy of 66.25% compared to 54.62% for the baseline. These findings highlight the value of architectural refinement and multi-dataset training for clinically viable ECG classification. These findings

support further development of multimodal, cross-dataset diagnostic AI systems for real-world development.

Table of Contents

Abstract.....i

Introduction.....1

Literature Review.....2

Data.....2

Methods.....3

Results.....6

Analysis and Interpretation.....6

Conclusion.....8

Directions for Future Work.....9

References.....11

Appendix.....12

Introduction

Cardiovascular disease remains the leading cause of mortality worldwide, with cardiac arrhythmias contributing significantly to morbidity, hospitalizations, and sudden cardiac death. Electrocardiography (ECG) remains the primary diagnostic modality for arrhythmia detection due to its accessibility, cost-effectiveness, and diagnostic specificity. In recent years, deep learning models have shown remarkable performance in automating ECG interpretation, achieving cardiologist level accuracy in identifying common arrhythmias, such as atrial fibrillation, bundle, branch walks, and premature ventricular contractions.

Despite this, many ECG classification models are limited by their ability to generalize beyond the training data set. Variations in ECG acquisition systems and annotation protocols can lead to domain shifts, which degrade performance when applied to new clinical settings. This study integrates two heterogeneous, clinically relevant datasets (PTB-XL and the Lobachevsky University databases) to enhance model robustness and reduce dataset-specific bias. Building upon a previously developed CNN architecture, this research introduces a more discriminative CNN

model designed to improve arrhythmia classification performance. The evaluation focuses on clinically meaningful metrics, providing a comprehensive assessment of diagnostic reliability.

Literature Review

The recent advent of deep learning introduced architectures like 1D convolutional neural networks and recurrent neural networks capable of learning hierarchical and temporal presentations directly from raw ECG waveforms. For example, Hannun et al. (2019) demonstrated cardiologist-level detection from single lead ECGs using deep CNNs, while Yildirim (2018) applied CNN-LSTM hybrids for improved temporal rhythm modeling. Ribeiro et al. (2020) expanded on Hannun et al. by applying a deep residual CNN to 12-lead ECGs, improving diagnostic breadth across multiple arrhythmia categories.

Despite this literature, cross-dataset generalization remains underexplored. Studies that report results on a single source dataset often lead to poor real-world transferability.

Data

The PTB-XL dataset, published by Wagner et al. (2020) and hosted on PhysioNet, comprises 21,837 12-lead ECG recordings from 18,885 patients. Each ECG is 10 seconds in duration and

sampled at 500 Hz. Labels follow the SCP-ECG taxonomy, covering 71 diagnostic classes grouped into five categories: *Normal ECG, Myocardial Infarction, Conduction Disturbance, Hypertrophy, and ST/T Changes*. The dataset includes both demographic metadata and physician-verified labels.

The Lobachevsky dataset contains multi-lead ECG signals collected from a different patient population using distinct acquisition hardware and clinical protocols. This introduces variability in noise levels, demographic features, and recording conditions. Such heterogeneity is valuable for examining cross-dataset generalization and identifying whether models training on PTB-XL can maintain strong performance across external ECG sources.

Methods

Both datasets were preprocessed to standardize input length, normalize signal amplitude, and ensure consistent lead ordering. The final merged training set contained ECG segments from both sources, enabling broad distributional coverage.

First Model

The baseline model was constructed as a conventional 1D CNN designed to extract local temporal features from ECG

waveforms. The architecture begins with an initial Conv1D layer applied directly to the raw input sequence, using moderately sized kernels to capture short range morphological patterns. Each convolutional block is followed by batch normalization, which stabilizes training by normalizing intermediate feature distributions and a Relu activation function to introduce non-linearity.

Following the early convolutional layers, the model incorporates max polling layers to down sample the temporal dimension and reduce computational cost while retaining salient features. As the network progresses, drop out layers are placed after convolutional or dense blocks to mitigate over-fitting, especially important when training on heterogeneous ECG data from PTB-XL and Lobachevsky University.

After the feature extraction stage, the model transitions into a fully connected classification head, consisting of a dense layer with Relu activation and an output layer using softmax to generate class probabilities. Overall, the architecture represents a straightforward CNN pipeline that effectively captures localized ECG features, but lacks the depth and representational power needed to fully model complex inter-class differences and long range dependencies.

Second Model

The second model expands substantially on the baseline designed by introducing a deeper architectural hierarchy and

multiple residual connections inspired by ResNet-style networks. The architecture begins with the initial Conv1D block similar to the baseline model, but immediately scales the number of filters to increase the representational capacity.

The core of the model consists of residual blocks, each containing two sequential Conv1D layers with batch normalization and Relu activations. Within each block, the input is added directly to the output of the convolutional layers, forming a skip connection. This mechanism mitigates vanishing gradients, stabilizes training across many layers, and allows the model to learn both shallow and deep feature transformations simultaneously.

As the network deepens, the model progressively increases the number of filters while occasionally applying strided convolutions to reduce the temporal dimension without information loss associated with max pooling. This approach helps preserve fine-grained waveform structure. Regularization is further strengthened through strategically placed dropout layers, which reduce co-adaptation of filters and improve generalization across data sets with different acquisition conditions.

Before classifications, the model employs a global average pooling layer to aggregate temporal features into a compact representation. This pooling strategy reduces the number of trainable parameters compared to dense layers and has been shown to improve robustness in noisy biomedical signals. The final

classification layer uses a softmax output to generate probabilities across the diagnostic classes.

Collectively, this enhanced architecture provides a significantly more expressive feature extractor capable of capturing nuanced arrhythmia morphology and long range dependencies. Its combination of depth, residual connectivity, and strong normalization explains its improved performance across test accuracy, ROC/AUC, and precision recall metrics.

Both models' exact architectures are outlined in Appendix 1.

Results

First Model

Test Accuracy: 0.5462

Second Model

Test Accuracy: 0.6624

Analysis and Interpretation

The baseline CNN achieved a test accuracy of 54.62%, indicating that while the model successfully learned the broad ECG waveform patterns, it struggled to generalize across the full set of diagnostic categories. Examination of the confusion matrix revealed that the baseline model frequently misclassified arrhythmias with

subtle morphological differences, particularly among rhythm types, characterized by overlapping, QRS and P wave structures. Although the ROC curves for several classes showed moderate discriminatory ability, the overall AUC values indicated that the baseline model did not consistently separate positive and negative instances across all diagnostic labels. Precision recall curves further confirm this limitation, showing lower precision in classes with higher intra-class variability.

In contrast, the improved residual CNN demonstrated substantial performance gains, achieving a test accuracy of 66.25%. This improvement reflects the enhanced model's ability to extract more discriminative temporal and morphological features from multi source ECG signals. The confusion matrix for the residual CNN showed notably reduced misclassification in congested diagnostic categories, especially where PTB-XL and Lobachevsky records differed in signal quality or patient demographics. ROC curves for the improved model displayed consistently steeper trajectories, and the AUC scores were higher across nearly all classes, indicating superior threshold-independent classification performance. Precision-recall curves also showed marked improvements, with higher precision at equivalent recall levels, demonstrating stronger robustness in handling class imbalance.

Conclusion

This study examined performance of two convolutional neural network architectures: one baseline model and one enhanced residual model, which was trained on ECG data sourced from the PTB-XL and Lobachevsky University databases. By evaluating both models using confusion matrices, ROC curves, AUC scores, and precision-recall curves, this research demonstrated the critical importance of architectural depth, residual connections, and strong regularization strategies for accurate arrhythmia classification in heterogeneous, multi-source datasets. The improved residual CNN achieved substantially higher accuracy and more consistent discriminatory performance, indicating that deeper hierarchical feature extraction is better suited to capturing the morphological complexity inherent in ECG signals.

The findings underscore the feasibility of deploying deep learning systems for automated arrhythmia detection across diverse clinical environments. They also emphasize that generalization remains a central challenge when combining these assets with differing acquisition protocols, noise characteristics, and demographic compositions. Overall, the study contributes to the growing body of work supporting the integration of advanced deep

learning models into clinical diagnostic pipelines, with potential benefits for early detection, triage efficiency, and patient outcomes.

Directions for Future Work

Future research should expand beyond the current residual CNN by exploring more advanced architectural families that can capture richer temporal patterns in ECG signals. This can include, but not limited to evaluating ResNet-1D blocks, Inception-1D modules, and XResNet-1D variants, all of which offer complementary benefits through deeper feature extraction, multi-scale convolutional pathways, and stabilized gradient flow. These architectures may substantially improve discrimination of complex arrhythmias, especially those involving subtle or overlapping wayform morphologies.

A second direction involves the development of lightweight temporal models suitable for real-time clinical deployment. Architectures based on 1D MobileNet and depth wise separable convolutions provide competitive accuracy with far lower computational cost, making them ideal for integration into wearable devices, telemetry systems, and bedside monitors. In parallel, temporal convolutional networks with dilated convolutions

should be examined for their ability to model long range dependencies without the overhead of recurrent layers.

Finally, future work should investigate alternative signal representations that capture both temporal and frequency domain characteristics. Transforming ECG waveforms into spectrogram-like formats using the short-time Fourier transform or the continuous wavelet transform may reveal useful frequency patterns and permit the application of powerful 2D CNN or hybrid architectures. These approaches collectively provide a promising path toward more expressive, efficient, and clinically generalizable arrhythmia detection systems.

References

1. Hannun, Awni Y., Pranav Rajpurkar, Masoumeh Haghpanahi, Geoffrey H. Tison, Codie Bourn, Mintu P. Turakhia & Andrew Y. Ng. 2019. "*Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network*." Nature Medicine. https://doi.org/10.1038/s41591-018-0268-3.

2. Yıldırım, Ö. 2018. "*A Novel Wavelet Sequence Based on Deep Bidirectional LSTM Network Model for ECG Signal Classification*." Computers in Biology and Medicine. doi:10.1016/j.compbiomed.2018.03.016.

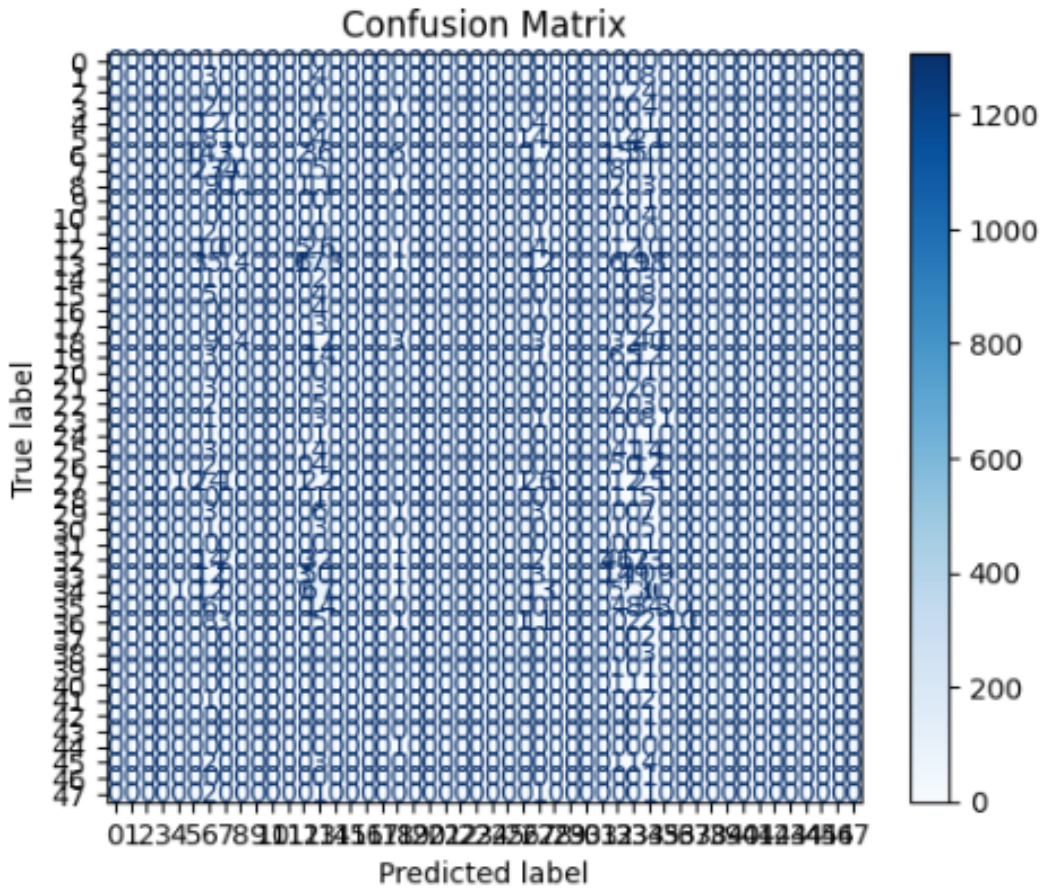
3. Ribeiro, Antonio H., Manoel Horta Ribeiro, Gabriela M. Paixão, et al. 2020. "Automatic Diagnosis of the 12-Lead ECG Using a Deep Neural Network." Nature Communications 11: 1760. https://www.nature.com/articles/s41467-020-15432-4.

Appendix

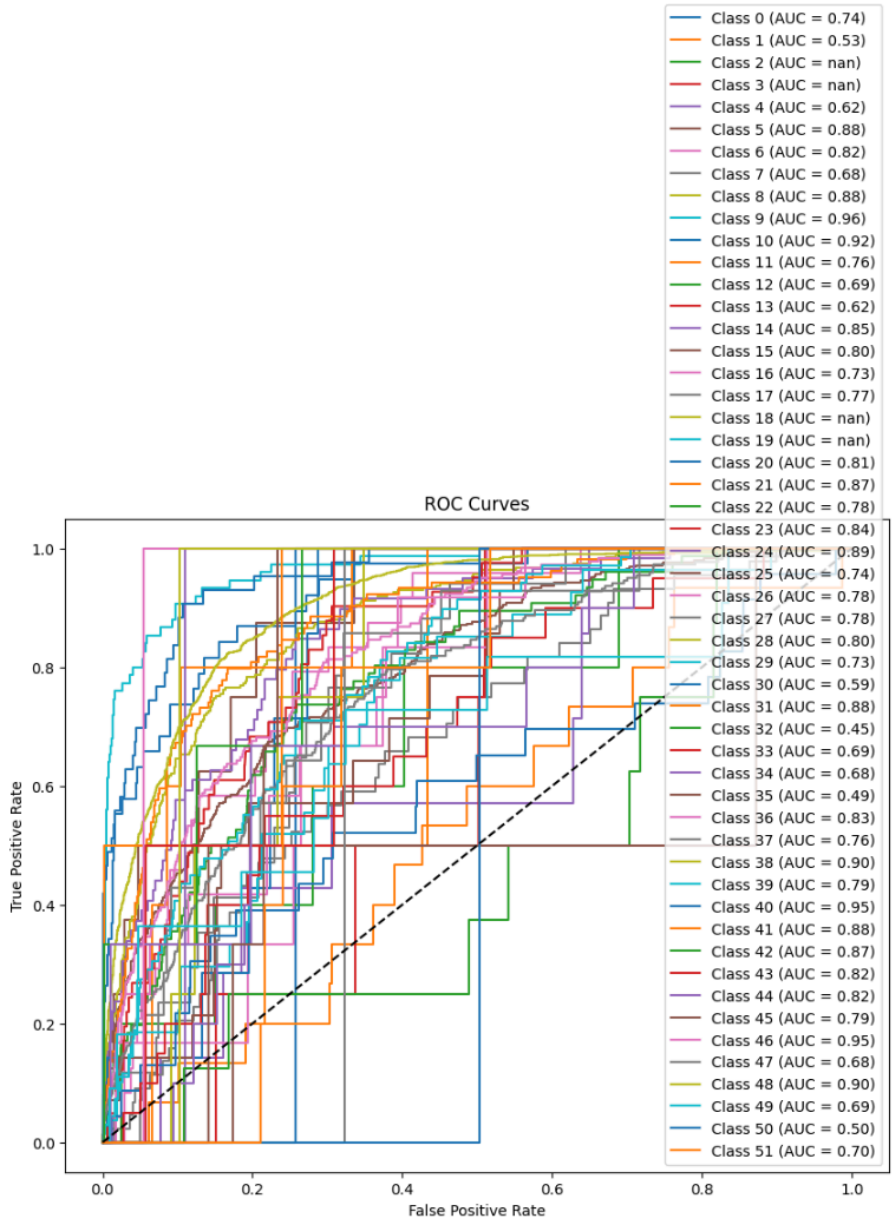
First Model Architecture

```
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv1d (Conv1D)             (None, 4994, 32)          2720

 batch_normalization (Batch  (None, 4994, 32)          128
 Normalization)

 max_pooling1d (MaxPooling1  (None, 1248, 32)          0
 D)

 conv1d_1 (Conv1D)           (None, 1244, 64)          10304

 batch_normalization_1 (Bat  (None, 1244, 64)          256
 chNormalization)

 max_pooling1d_1 (MaxPoolin  (None, 311, 64)           0
 g1D)

 conv1d_2 (Conv1D)           (None, 309, 128)          24704

 batch_normalization_2 (Bat  (None, 309, 128)          512
 chNormalization)

 max_pooling1d_2 (MaxPoolin  (None, 154, 128)          0
 g1D)

 dropout (Dropout)           (None, 154, 128)          0

 flatten (Flatten)           (None, 19712)             0

 dense (Dense)               (None, 128)               2523264

 dropout_1 (Dropout)         (None, 128)               0

 dense_1 (Dense)             (None, 52)                6708

=================================================================
Total params: 2568596 (9.80 MB)
Trainable params: 2568148 (9.80 MB)
Non-trainable params: 448 (1.75 KB)
_________________________________________________________________
```

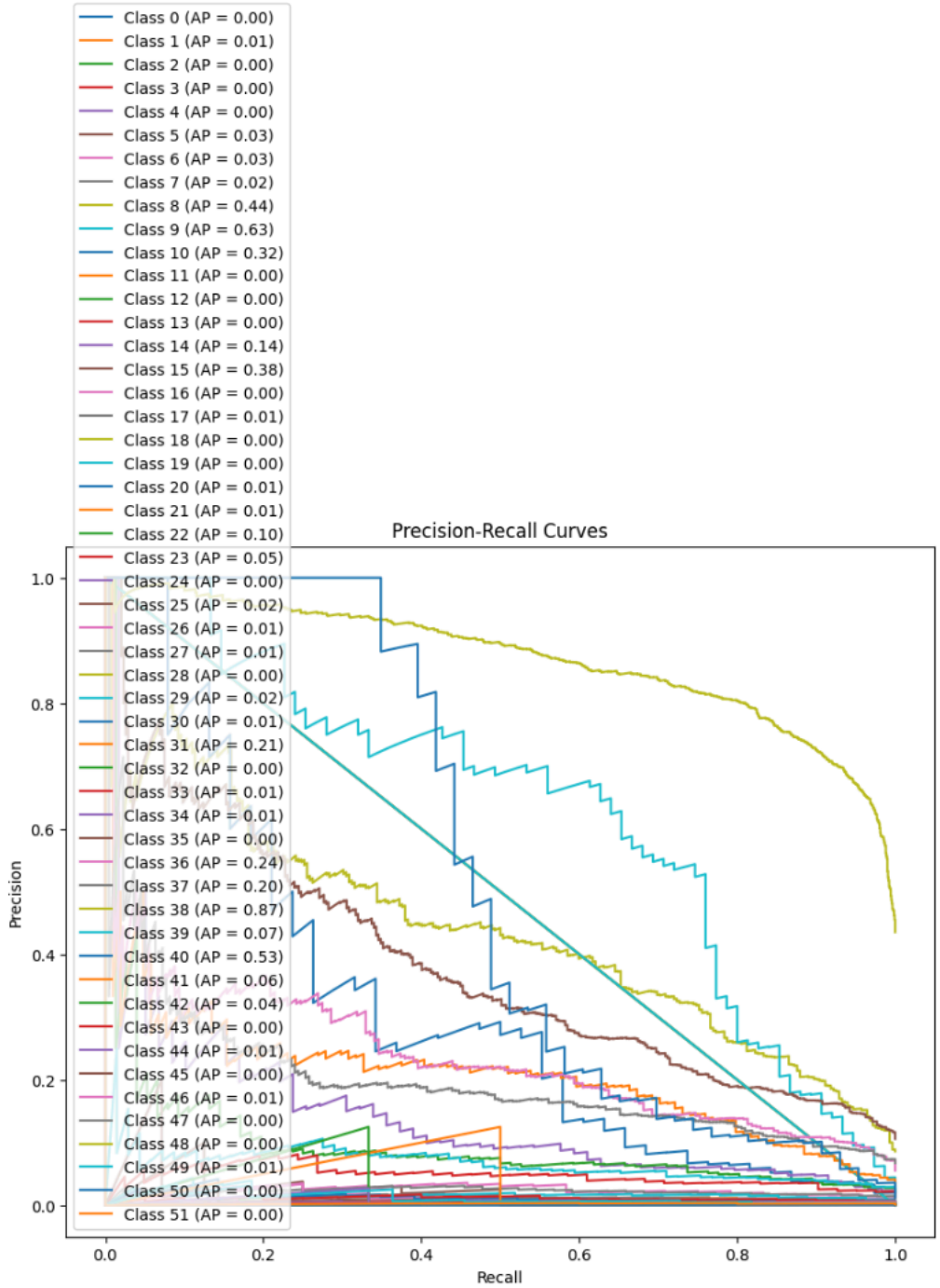
First Model Confusion Matrix



First Model ROC Curves (with AUC scores)



First Model Precision-Recall Curve

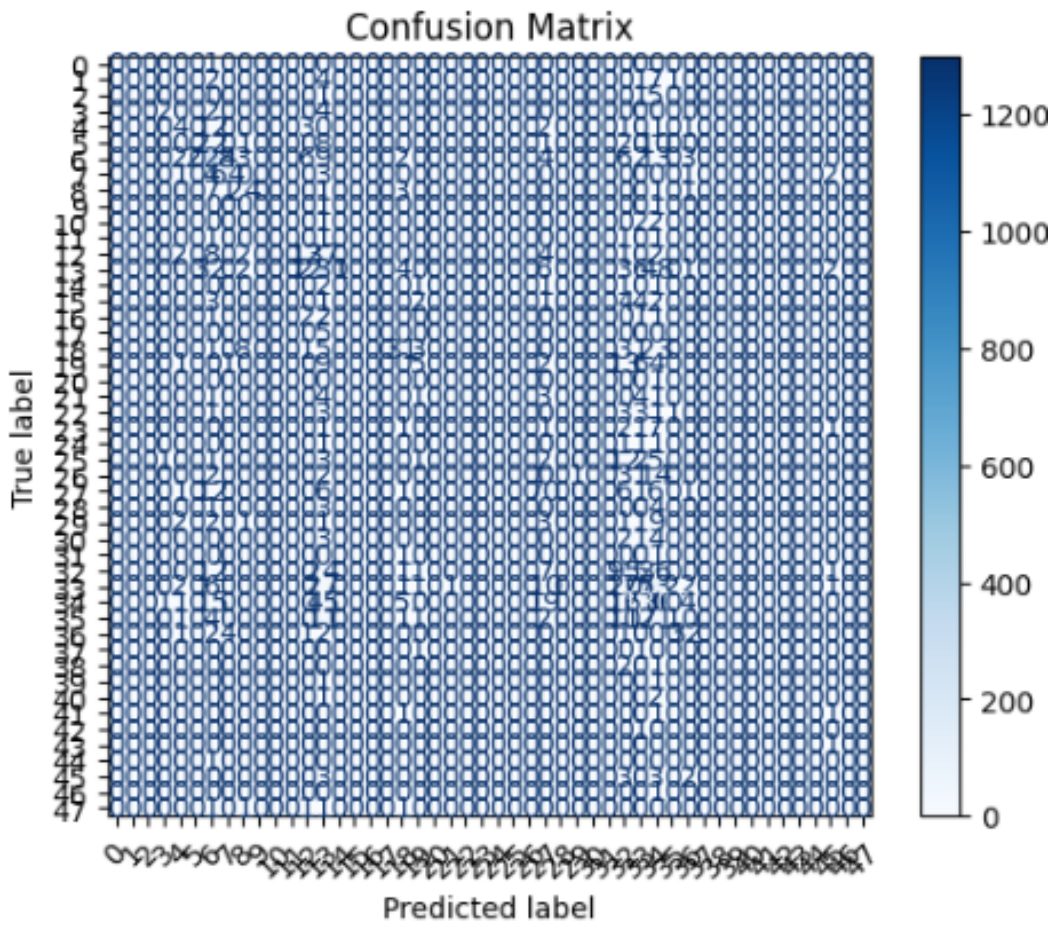


Second Model Architecture

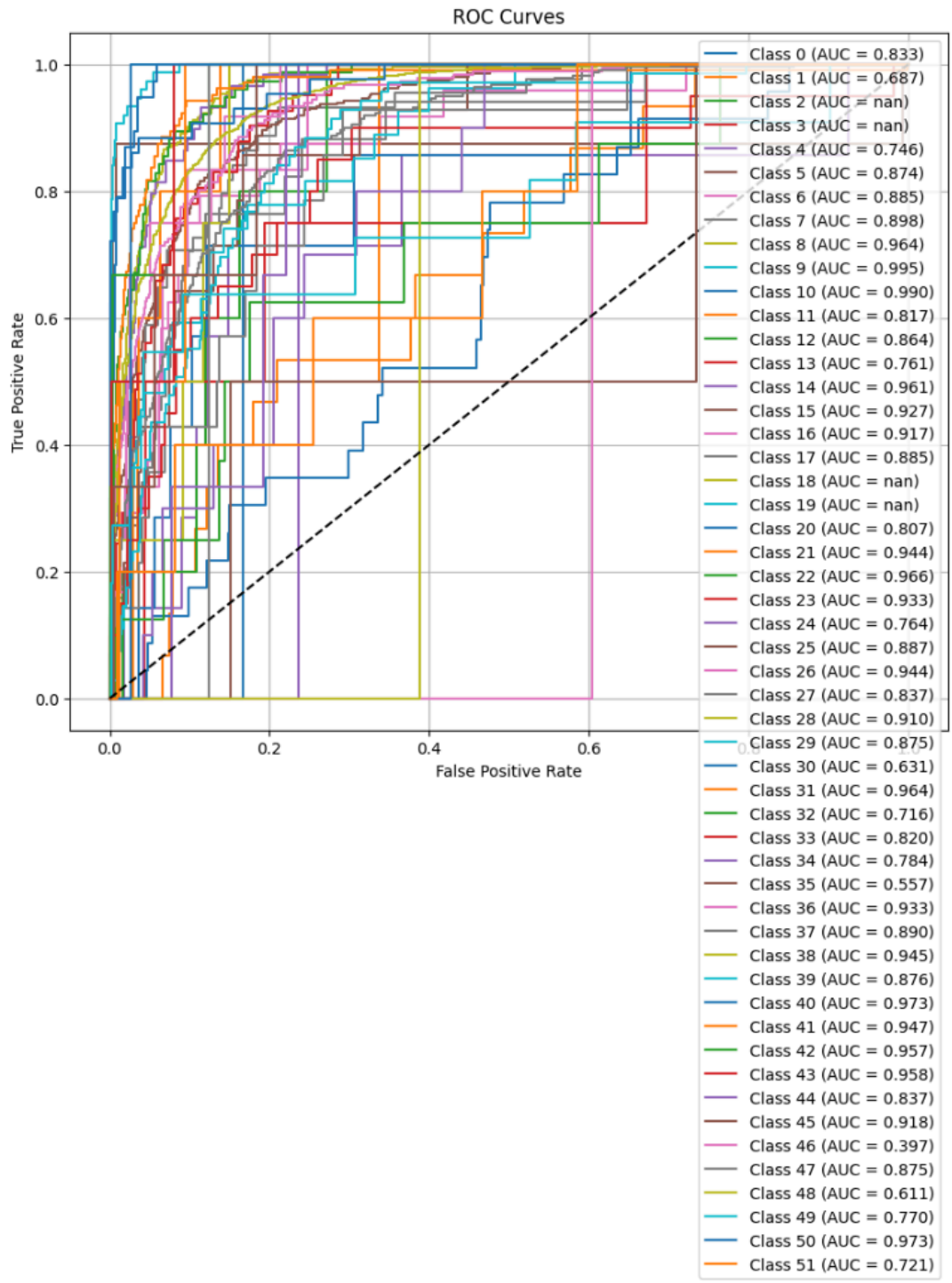
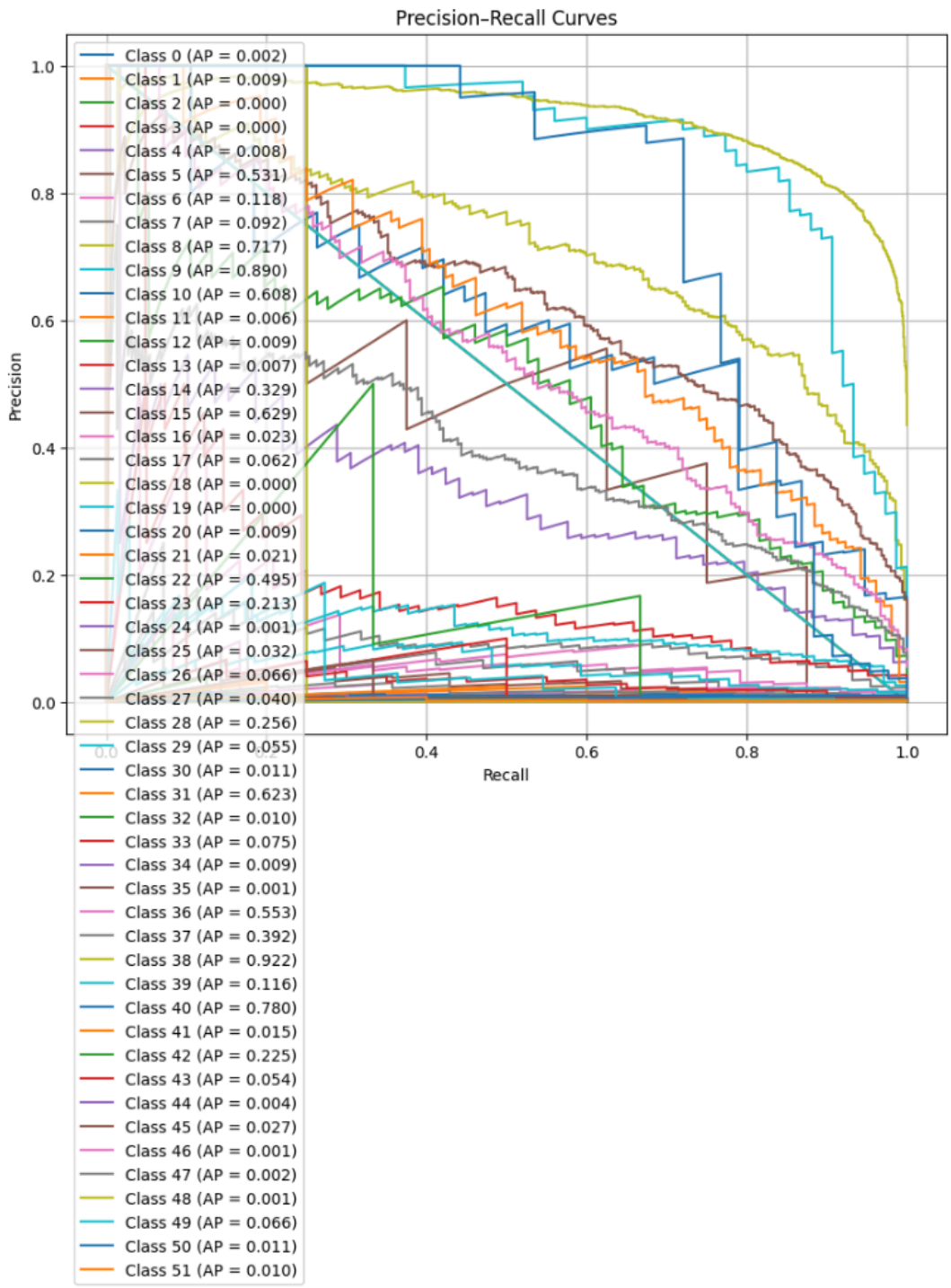
Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 3000, 12)]	0	[]
conv1d_3 (Conv1D)	(None, 3000, 64)	11584	['input_1[0][0]']
batch_normalization_3 (BatchNormalization)	(None, 3000, 64)	256	['conv1d_3[0][0]']
conv1d_4 (Conv1D)	(None, 3000, 64)	28736	['batch_normalization_3[0][0]']
batch_normalization_4 (BatchNormalization)	(None, 3000, 64)	256	['conv1d_4[0][0]']
activation (Activation)	(None, 3000, 64)	0	['batch_normalization_4[0][0]']
conv1d_5 (Conv1D)	(None, 3000, 64)	28736	['activation[0][0]']
batch_normalization_5 (BatchNormalization)	(None, 3000, 64)	256	['conv1d_5[0][0]']
add (Add)	(None, 3000, 64)	0	['batch_normalization_5[0][0]', 'batch_normalization_3[0][0]']
activation_1 (Activation)	(None, 3000, 64)	0	['add[0][0]']
dropout_2 (Dropout)	(None, 3000, 64)	0	['activation_1[0][0]']
conv1d_6 (Conv1D)	(None, 1500, 128)	41088	['dropout_2[0][0]']
batch_normalization_6 (BatchNormalization)	(None, 1500, 128)	512	['conv1d_6[0][0]']
activation_2 (Activation)	(None, 1500, 128)	0	['batch_normalization_6[0][0]']
conv1d_7 (Conv1D)	(None, 1500, 128)	82048	['activation_2[0][0]']
conv1d_8 (Conv1D)	(None, 1500, 128)	8320	['dropout_2[0][0]']
batch_normalization_7 (BatchNormalization)	(None, 1500, 128)	512	['conv1d_7[0][0]']
batch_normalization_8 (BatchNormalization)	(None, 1500, 128)	512	['conv1d_8[0][0]']
add_1 (Add)	(None, 1500, 128)	0	['batch_normalization_7[0][0]', 'batch_normalization_8[0][0]']
activation_3 (Activation)	(None, 1500, 128)	0	['add_1[0][0]']
dropout_3 (Dropout)	(None, 1500, 128)	0	['activation_3[0][0]']
conv1d_9 (Conv1D)	(None, 1500, 128)	82048	['dropout_3[0][0]']
batch_normalization_9 (BatchNormalization)	(None, 1500, 128)	512	['conv1d_9[0][0]']
activation_4 (Activation)	(None, 1500, 128)	0	['batch_normalization_9[0][0]']

```
 conv1d_10 (Conv1D)          (None, 1500, 128)            82048     ['activation_4[0][0]']

 batch_normalization_10 (Ba  (None, 1500, 128)            512       ['conv1d_10[0][0]']
 tchNormalization)

 add_2 (Add)                 (None, 1500, 128)            0         ['batch_normalization_10[0][0]
                                                                     ',
                                                                      'dropout_3[0][0]']

 activation_5 (Activation)   (None, 1500, 128)            0         ['add_2[0][0]']

 dropout_4 (Dropout)         (None, 1500, 128)            0         ['activation_5[0][0]']

 conv1d_11 (Conv1D)          (None, 750, 256)             98560     ['dropout_4[0][0]']

 batch_normalization_11 (Ba  (None, 750, 256)             1024      ['conv1d_11[0][0]']
 tchNormalization)

 activation_6 (Activation)   (None, 750, 256)             0         ['batch_normalization_11[0][0]
                                                                     ']

 conv1d_12 (Conv1D)          (None, 750, 256)             196864    ['activation_6[0][0]']

 conv1d_13 (Conv1D)          (None, 750, 256)             33024     ['dropout_4[0][0]']

 batch_normalization_12 (Ba  (None, 750, 256)             1024      ['conv1d_12[0][0]']
 tchNormalization)

 batch_normalization_13 (Ba  (None, 750, 256)             1024      ['conv1d_13[0][0]']
 tchNormalization)

 add_3 (Add)                 (None, 750, 256)             0         ['batch_normalization_12[0][0]
                                                                     ',
                                                                      'batch_normalization_13[0][0]
                                                                     ']

 activation_7 (Activation)   (None, 750, 256)             0         ['add_3[0][0]']

 dropout_5 (Dropout)         (None, 750, 256)             0         ['activation_7[0][0]']

 conv1d_14 (Conv1D)          (None, 750, 256)             196864    ['dropout_5[0][0]']

 batch_normalization_14 (Ba  (None, 750, 256)             1024      ['conv1d_14[0][0]']
 tchNormalization)

 activation_8 (Activation)   (None, 750, 256)             0         ['batch_normalization_14[0][0]
                                                                     ']

 conv1d_15 (Conv1D)          (None, 750, 256)             196864    ['activation_8[0][0]']

 batch_normalization_15 (Ba  (None, 750, 256)             1024      ['conv1d_15[0][0]']
 tchNormalization)

 add_4 (Add)                 (None, 750, 256)             0         ['batch_normalization_15[0][0]
                                                                     ',
                                                                      'dropout_5[0][0]']
 activation_9 (Activation)   (None, 750, 256)             0         ['add_4[0][0]']

 dropout_6 (Dropout)         (None, 750, 256)             0         ['activation_9[0][0]']

 global_average_pooling1d (  (None, 256)                  0         ['dropout_6[0][0]']
 GlobalAveragePooling1D)
 dense_2 (Dense)             (None, 256)                  65792     ['global_average_pooling1d[0][
                                                                     0]']
 dropout_7 (Dropout)         (None, 256)                  0         ['dense_2[0][0]']

 dense_3 (Dense)             (None, 52)                   13364     ['dropout_7[0][0]']
==================================================================================================
Total params: 1174388 (4.48 MB)
Trainable params: 1170164 (4.46 MB)
Non-trainable params: 4224 (16.50 KB)
__________________________________________________________________________________________________
```

Second Model Confusion Matrix



Second Model ROC Curves (and AUC scores)



Second Model Precision-Recall Curve